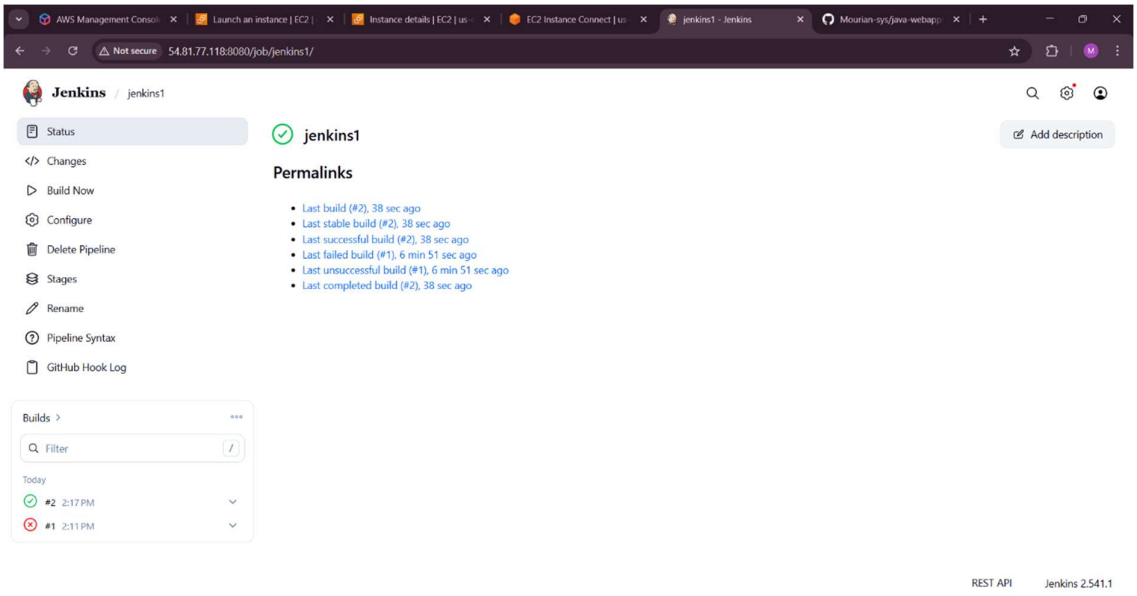
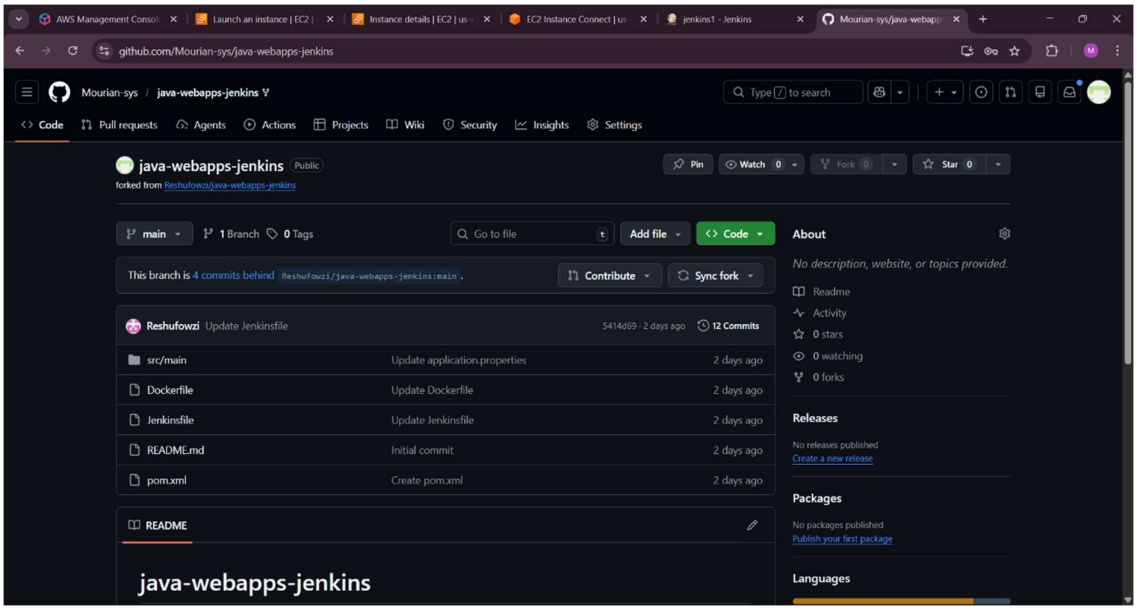
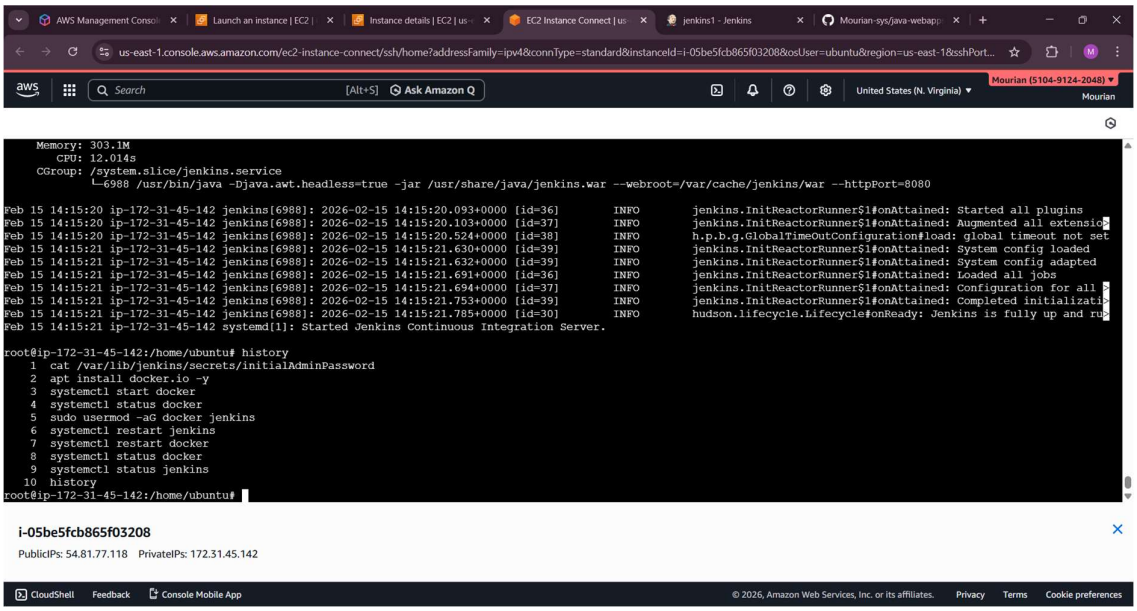
ASSIGNMENT 31

Jenkins with dockerfile




AWS Management Consol
Launch an instance | EC2 |
Instance details | EC2 | us-
EC2 Instance Connect | us-
jenkins1 - Jenkins
Mourian-sys/java-webapp
github.com/Mourian-sys/java-webapps-jenkins
Mourian-sys / java-webapps-jenkins
Type / to search
Code
Pull requests
Agents
Actions
Projects
Wiki
Security
Insights
Settings
java-webapps-jenkins Public
forked from Reshufowzi/java-webapps-jenkins
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
t
Add file
Code
This branch is 4 commits behind Reshufowzi/java-webapps-jenkins:main.
Contribute
Sync fork
Reshufowzi Update Jenkinsfile
5414d69 · 2 days ago
12 Commits
src/main
Update application.properties
2 days ago
Dockerfile
Update Dockerfile
2 days ago
Jenkinsfile
Update Jenkinsfile
2 days ago
README.md
Initial commit
2 days ago
pom.xml
Create pom.xml
2 days ago
README
java-webapps-jenkins
About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Languages


AWS Management Consol
Launch an instance | EC2 |
Instance details | EC2 | us-
EC2 Instance Connect | us-
jenkins1 - Jenkins
Mourian-sys/java-webapp
us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-05be5fcb865f03208&osUser=ubuntu&region=us-east-1&sshPort...
aws
Search
[Alt+S]
Ask Amazon Q
United States (N. Virginia)
Mourian (5104-9124-2048)
Mourian
Memory: 303.1M
CPU: 12.014s
CGroup: /system.slice/jenkins.service
└─6988 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080
Feb 15 14:15:20 ip-172-31-45-142 jenkins[6988]: 2026-02-15 14:15:20.093+0000 [id=36] INFO jenkins.InitReactorRunner$1#onAttained: Started all plugins
Feb 15 14:15:20 ip-172-31-45-142 jenkins[6988]: 2026-02-15 14:15:20.103+0000 [id=37] INFO jenkins.InitReactorRunner$1#onAttained: Augmented all extensio
Feb 15 14:15:20 ip-172-31-45-142 jenkins[6988]: 2026-02-15 14:15:20.524+0000 [id=38] INFO h.p.b.g.GlobalTimeOutConfiguration#load: global timeout not set
Feb 15 14:15:21 ip-172-31-45-142 jenkins[6988]: 2026-02-15 14:15:21.630+0000 [id=39] INFO jenkins.InitReactorRunner$1#onAttained: System config loaded
Feb 15 14:15:21 ip-172-31-45-142 jenkins[6988]: 2026-02-15 14:15:21.632+0000 [id=39] INFO jenkins.InitReactorRunner$1#onAttained: System config adapted
Feb 15 14:15:21 ip-172-31-45-142 jenkins[6988]: 2026-02-15 14:15:21.691+0000 [id=36] INFO jenkins.InitReactorRunner$1#onAttained: Loaded all jobs
Feb 15 14:15:21 ip-172-31-45-142 jenkins[6988]: 2026-02-15 14:15:21.694+0000 [id=37] INFO jenkins.InitReactorRunner$1#onAttained: Configuration for all
Feb 15 14:15:21 ip-172-31-45-142 jenkins[6988]: 2026-02-15 14:15:21.753+0000 [id=39] INFO jenkins.InitReactorRunner$1#onAttained: Completed initializati
Feb 15 14:15:21 ip-172-31-45-142 jenkins[6988]: 2026-02-15 14:15:21.785+0000 [id=30] INFO hudson.lifecycle.Lifecycle#onReady: Jenkins is fully up and ru
Feb 15 14:15:21 ip-172-31-45-142 systemd[1]: Started Jenkins Continuous Integration Server.
root@ip-172-31-45-142:/home/ubuntu# history
1 cat /var/lib/jenkins/secrets/initialAdminPassword
2 apt install docker.io -y
3 systemctl start docker
4 systemctl status docker
5 sudo usermod -aG docker jenkins
6 systemctl restart jenkins
7 systemctl restart docker
8 systemctl status docker
9 systemctl status jenkins
10 history
root@ip-172-31-45-142:/home/ubuntu#
i-05be5fcb865f03208
PublicIPs: 54.81.77.118 PrivateIPs: 172.31.45.142
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

Jenkins Setup Using Docker with GitHub Fork, JDK, Maven, and Webhook Integration

Step 1: Create Jenkins Using Docker

First, Jenkins is created using Docker. Docker allows Jenkins to run inside a container environment. This makes Jenkins easy to install, manage, and isolate from the main system. After creating the Jenkins container, Jenkins runs on port 8080. The Jenkins dashboard is accessed using the system IP address and port 8080 through a web browser.

When Jenkins starts for the first time, it asks for an administrator password. After entering the password, Jenkins allows installation of suggested plugins. Then, an admin username and password are created. After completing these steps, the Jenkins dashboard opens successfully.

Step 2: Fork Repository from GitHub

Next, an existing repository is forked from another GitHub account. Forking creates a copy of the original repository in the user's own GitHub account. This allows the user to make changes without affecting the original project. The forked repository is used to connect with Jenkins for automation.

Forking is important because Jenkins needs access to the user's own repository to perform build and automation tasks.

Step 3: Install Required Plugins Inside Jenkins

After logging in to Jenkins, required plugins are installed. This is done by going to the "Manage Jenkins" section and opening the plugins manager. The following plugins are installed:

- Git Plugin, which allows Jenkins to connect with GitHub repositories
- GitHub Plugin, which enables webhook integration
- Pipeline Plugin, which allows creation of automated pipelines
- Maven Integration Plugin, which supports building Java projects using Maven

These plugins extend Jenkins functionality and allow integration with GitHub and Java tools.

Step 4: Configure JDK17 Inside Jenkins

To build Java projects, JDK17 is configured inside Jenkins. This is done in the Global Tool Configuration section. A new JDK installation is added and named JDK17. This configuration allows Jenkins to use Java 17 during the build process.

JDK is required because Java projects need the Java Development Kit for compilation and execution.

Step 5: Configure Maven3 Inside Jenkins

Next, Maven3 is configured in the Global Tool Configuration section. A new Maven installation is added and named Maven3. Maven is a build automation tool used for compiling, testing, and packaging Java applications.

By configuring Maven inside Jenkins, Jenkins can automatically build Java projects using Maven.

Step 6: Add GitHub Token for Secure Access

To securely connect Jenkins with the forked GitHub repository, a GitHub Personal Access Token is generated. This token acts as a secure password. The token is added inside Jenkins credentials section.

Adding credentials ensures Jenkins can access the GitHub repository securely without authentication errors.

Step 7: Create Pipeline Job in Jenkins

After configuring tools and credentials, a pipeline job is created in Jenkins. A new pipeline project is created from the Jenkins dashboard. The forked GitHub repository is connected to the pipeline.

The pipeline defines the automated steps such as fetching code, building the project, testing, and packaging the application. This allows Jenkins to automate the entire software build process.

Step 8: Configure GitHub Webhook

To enable automatic build triggering, a webhook is configured in the forked GitHub repository. The webhook connects GitHub and Jenkins using the Jenkins webhook URL.

Whenever code changes are pushed to the forked repository, GitHub sends a notification to Jenkins. Jenkins automatically starts the pipeline without manual intervention.

This enables continuous integration.

Step 9: Automatic Build and Verification

After completing all configurations, Jenkins automatically detects code changes from the forked repository. Jenkins downloads the latest code, builds the project using Maven, runs tests, and packages the application.

The build result is displayed on the Jenkins dashboard. If successful, Jenkins shows a successful build status. This confirms that Jenkins is properly integrated with Docker, GitHub, JDK, and Maven.

Conclusion:

Jenkins is successfully deployed using Docker container. A repository is forked from GitHub and connected to Jenkins. Required plugins are installed inside Jenkins. JDK17 and Maven3 are

configured using Global Tool Configuration. GitHub credentials and webhook are added for secure and automatic integration. A pipeline job is created to automate the Java project build process. Jenkins now automatically builds and tests the project whenever changes are pushed to the forked repository. This setup demonstrates continuous integration using Jenkins, Docker, and GitHub.